We can improve this code. Rather than instantiating `ElasticsearchIndicesClient` separately, we can instead use a builder, as shown in the following listing.

Listing C.7 Creating an index using the builder pattern

```
/**
* A method to create the index using Builder pattern
* @param indexName
* @throws IOException
*/
public void createIndexUsingBuilder(String indexName) throws IOException {

CreateIndexResponse createIndexResponse  = this.elasticsearchClient
  .indices().create(new CreateIndexRequest.Builder()
  .index(indexName)
  .build());

System.out.println("Index created successfully using
        Builder"+createIndexResponse);
}
```

We do not create `ElasticsearchIndicesClient` in this method. Instead, we pass in the request object (as a builder) to the `create()` method (the `indices()` method fetches the `ElasticsearchIndicesClient` on which the `create()` method is called behind the scenes).

We can go further—we can use a lambda function to make the code even more concise.

Listing C.8 Creating an index using Lambda expressions

```
/**
* A method to create an index using Lambda expression
* @param indexName
* @throws IOException
*/
public void createIndexUsingLambda(String indexName) throws IOException {

CreateIndexResponse createIndexResponse =
this.elasticsearchClient.indices().create(
                  request -> request.index(indexName)
        );

System.out.println("Index created successfully using Lambda"
        +createIndexResponse);
}
```

The bold portion of the code is the lambda expression. It says that given a request (which is a `CreateIndexRequest.Builder()` type object), create an index using the `ElasticsearchIndicesClient` client.